Gradient Boosting Machine (GBM)

Aymen Negadi

July 2026

“‘latex Gradient Boosting Machine (GBM)

1 Introduction

Gradient Boosting Machine (GBM) is a powerful supervised machine learning algorithm used for both classification and regression tasks. It belongs to the family of ensemble learning methods, where multiple weak learners are combined to produce a strong predictive model.

Unlike Random Forest, which builds many independent decision trees in parallel, Gradient Boosting constructs trees sequentially. Each new tree attempts to correct the prediction errors made by the previous ensemble. Through this iterative process, the overall model gradually improves its predictive performance.

GBM is widely recognized for its high accuracy and is the foundation of several advanced algorithms such as XGBoost, LightGBM, and CatBoost.

2 Motivation

A single decision tree is easy to interpret but often suffers from high variance or high bias depending on its complexity. Instead of relying on one large tree, Gradient Boosting builds many small trees, each responsible for correcting the mistakes of the previous ones.

The central idea is simple:

> Each new tree learns from the errors of the current model and improves the overall prediction.

Consequently, the final model becomes much more accurate than any individual tree.

3 Boosting Concept

Boosting is an ensemble learning technique that combines several weak learners into one strong learner.

A weak learner is a model that performs only slightly better than random guessing.

In Gradient Boosting, weak learners are typically shallow decision trees, often called regression trees.

Suppose three trees are built sequentially:

$$T_1, \quad T_2, \quad T_3$$

The final prediction is obtained by summing the predictions of all trees:

$$F(x) = T_1(x) + T_2(x) + T_3(x).$$

More generally, after M trees,

$$F_M(x) = \sum_{m=1}^{M} T_m(x).$$

Each tree contributes only a small correction to the previous model.

4 Sequential Learning

The learning process follows four main steps:

1. Build the first decision tree.
2. Compute the prediction errors.
3. Train a new tree to predict these errors.
4. Add the new tree to the existing model.

This process continues until the desired number of trees has been generated. Unlike bagging methods, every tree depends on the trees built before it.

5 Residual Errors

The prediction error of each observation is called the residual.

For a training example,

$$y_i$$

is the true value, while

$$\hat{y}_i$$

is the predicted value.

The residual is

$$r_i = y_i - \hat{y}_i.$$

Instead of predicting the original target values again, the next tree predicts these residuals.

Example

Consider the following regression problem.

Actual	Prediction	Residual
10	8	2
20	17	3
30	35	-5

The second tree is trained using the residual values

$$2,\ 3,\ -5$$

instead of

$$10,\ 20,\ 30.$$

This enables the model to gradually reduce its prediction error.

6 Gradient Descent

The objective of Gradient Boosting is to minimize a loss function.

Instead of directly minimizing the error, the algorithm computes the negative gradient of the loss function.

The gradient indicates the direction in which the loss decreases most rapidly.

Hence the name **Gradient Boosting**.

At iteration m,

$$r_i^{(m)} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}.$$

These negative gradients become the target values for the next tree.

7 Loss Functions

Different problems require different loss functions.

For regression, one common choice is the Mean Squared Error (MSE):

$$L(y, \hat{y}) = (y - \hat{y})^2.$$

For binary classification, the logistic loss is commonly used:

$$L(y, p) = - \left[y \log(p) + (1 - y) \log(1 - p) \right].$$

The choice of the loss function determines how the residuals are computed during training.

8 Learning Rate

Each tree contributes only a fraction of its prediction.

The updated model is

$$F_m(x) = F_{m-1}(x) + \eta T_m(x),$$

where

$$0 < \eta \leq 1$$

is called the learning rate.

A small learning rate slows the learning process but often improves generalization and reduces overfitting.

Typical values are

$$0.1, \quad 0.05, \quad 0.01.$$

9 Number of Trees

The parameter M represents the number of boosting iterations.

Increasing the number of trees generally improves performance until overfitting begins.

A small learning rate usually requires more trees.

10 Tree Depth

The individual trees used in GBM are intentionally shallow.

Typical depths range from

$$3$$

to

$$8.$$

Shallow trees capture simple patterns while reducing model complexity.

11 Subsampling

Some Gradient Boosting implementations train each tree using only a random subset of the training data.

If

$$subsample = 0.8,$$

then only 80% of the training examples are selected for each iteration.

Subsampling reduces variance and improves generalization.

12 Training Algorithm

The training procedure can be summarized as follows:

1. Initialize the model.
2. Compute predictions.
3. Calculate residual errors.
4. Fit a decision tree to the residuals.
5. Update the model.
6. Repeat until the stopping criterion is reached.

13 Worked Numerical Example

Suppose the initial prediction of a model is

$$80.$$

The newly trained tree predicts a correction of

$$10.$$

Using a learning rate of

$$\eta = 0.1,$$

the updated prediction becomes

$$80 + 0.1 \times 10 = 81.$$

Instead of making a large correction, GBM updates the prediction gradually.

14 Classification Example

Consider the following dataset.

Hours Studied	Result
1	Fail
2	Fail
3	Pass
5	Pass

Assume the first tree predicts

Hours	Prediction
1	Fail
2	Fail
3	Fail
5	Pass

The observation corresponding to three hours is incorrectly classified.
The second tree focuses on correcting this mistake.
After combining both trees, the final predictions become

Hours	Prediction
1	Fail
2	Fail
3	Pass
5	Pass

The overall classification accuracy improves.

15 Advantages

Gradient Boosting offers several important advantages:

- Excellent predictive accuracy.
- Handles nonlinear relationships.
- Applicable to both regression and classification.
- Flexible choice of loss functions.
- Effective for structured and tabular datasets.
- Supports feature importance estimation.

16 Disadvantages

Despite its strengths, GBM has several limitations.

- Computationally expensive.
- Longer training time.
- Sensitive to hyperparameter selection.
- Can overfit if too many trees are used.
- Less interpretable than a single decision tree.

17 Comparison with Decision Tree

Property	Decision Tree	GBM
Number of Trees	1	Many
Learning Strategy	Independent	Sequential
Accuracy	Moderate	High
Training Speed	Fast	Slower
Overfitting Control	Limited	Better with tuning

18 Comparison with Random Forest

Property	Random Forest	GBM
Training	Parallel	Sequential
Technique	Bagging	Boosting
Goal	Reduce variance	Reduce bias
Training Speed	Faster	Slower
Accuracy	High	Often Higher
Hyperparameter Tuning	Easier	More Sensitive

19 Comparison with AdaBoost

Property	AdaBoost	GBM
Error Handling	Sample weights	Residuals/Gradients
Optimization	Weight updates	Gradient descent
Flexibility	Limited	High
Loss Functions	Few	Many
Performance	Good	Excellent

20 Applications

Gradient Boosting is widely used in many practical domains.

- Medical diagnosis.
- Credit scoring.
- Fraud detection.
- Customer churn prediction.
- Marketing analytics.
- Recommendation systems.
- Insurance risk assessment.
- Financial forecasting.

21 Summary

Gradient Boosting Machine is one of the most successful ensemble learning algorithms. It constructs a sequence of weak decision trees, where each tree learns to correct the errors made by the previous ensemble. By minimizing a differentiable loss function through gradient descent, GBM gradually improves prediction accuracy. Although it requires careful hyperparameter tuning and longer training times, its predictive performance makes it one of the most widely used algorithms in modern machine learning. “‘